

RELATÓRIO DE APRENDIZAGEM

Aula 07 – Expressões, Atribuição e Controle

Paradigmas de Linguagens de Programação

Engenharia de Software

02 de outubro de 2026

Este relatório registra os conhecimentos desenvolvidos na Aula 07, cujo tema foi “Expressões, Atribuição e Controle”. O conteúdo foi estudado com base no material da aula e nos Capítulos 7 e 8 de Conceitos de Linguagens de Programação, de Robert Sebesta, 11ª edição.

A aula teve caráter exploratório: antes de executar os exemplos, a proposta era prever o resultado e depois verificar o comportamento. Dessa forma, o estudo não ficou limitado à memorização de comandos, mas buscou compreender as regras que fazem uma linguagem interpretar determinado código de uma maneira específica.

Objetivo e ligação com o mundo real

Objetivo

O objetivo desta aula é entender **como as linguagens avaliam expressões e controlam o fluxo de execução**, para que o seu código faça exatamente o que parece fazer, **sem surpresas** de precedência, conversão ou desvio.

Para o engenheiro de software, isso significa:

- ler código alheio sem ser enganado pela indentação ou pela precedência
- evitar conversões implícitas perigosas (== do JS e do PHP)
- escolher a estrutura de controle mais legível para cada caso

Quando o fluxo sai do controle

2014 · "goto fail" (Apple)

Uma linha `goto fail`; duplicada pulava a verificação de assinatura: iOS e macOS aceitavam certificados falsos.

2008 · Tocadores Zune (Microsoft)

No último dia de um ano bissexto, um `while` nunca terminava, e todos os aparelhos travaram.

PHP · comparação com ==

`"0e4..." == "0e8..."` é verdadeiro: logins que comparavam hashes com == aceitavam senhas erradas.

Sebesta, 11ª ed. - Caps. 7 e 8 - p. 299-361

1. Objetivos da aula e relação com problemas reais

O objetivo principal foi compreender como as linguagens de programação avaliam expressões e controlam o fluxo de execução. Essa compreensão é necessária para que o código faça aquilo que o programador realmente pretende, evitando surpresas causadas por precedência, conversões automáticas ou desvios de fluxo.

Um ponto importante foi perceber que a aparência do código nem sempre representa sua interpretação real. A indentação pode sugerir uma associação entre comandos que o compilador não utiliza, enquanto uma conversão automática pode alterar o tipo de um valor antes de uma operação. Por isso, a leitura de código precisa considerar as regras formais da linguagem.

O material apresentou exemplos reais para demonstrar a importância do tema. O caso “goto fail”, relacionado à Apple, mostrou como um desvio duplicado poderia fazer uma verificação importante deixar de ser executada. Também foi citado o caso dos tocadores Zune, em que um laço sem progresso levou os aparelhos a travarem em uma situação específica.

Outro exemplo envolve comparações com `==` em PHP. Valores que se parecem com números em notação científica podem ser tratados como números durante uma comparação, produzindo um resultado diferente do esperado. Esses exemplos mostram que detalhes de expressões e controle podem se transformar em problemas de segurança ou disponibilidade.

Objetivo e ligação com o mundo real

Objetivo

O objetivo desta aula é entender **como as linguagens avaliam expressões e controlam o fluxo de execução**, para que o seu código faça exatamente o que parece fazer, **sem surpresas** de precedência, conversão ou desvio.

Para o engenheiro de software, isso significa:

- ler código alheio sem ser enganado pela indentação ou pela precedência
- evitar conversões implícitas perigosas (`==` do JS e do PHP)
- escolher a estrutura de controle mais legível para cada caso

Quando o fluxo sai do controle

2014 · "goto fail" (Apple)

Uma linha `goto fail;` duplicada pulava a verificação de assinatura: iOS e macOS aceitavam certificados falsos.

2008 · Tocadores Zune (Microsoft)

No último dia de um ano bissexto, um laço `while` nunca terminava, e todos os aparelhos travaram.

PHP · comparação com `==`

`"0e4..." == "0e8..."` é verdadeiro: logins que comparavam hashes com `==` aceitavam senhas erradas.

Sebesta, 11ª ed. - Caps. 7 e 8 - p. 299-361

2. Expressões aritméticas, precedência e associatividade

As expressões aritméticas foram usadas para mostrar que a ordem de avaliação é uma característica de projeto da linguagem. Em $a + b * c$, com $a = 3$, $b = 4$ e $c = 5$, o resultado esperado em linguagens com precedência convencional é 23, porque a multiplicação é realizada antes da adição.

Além da precedência, existe a associatividade. Quando operadores possuem a mesma prioridade, a linguagem precisa definir se eles são agrupados da esquerda para a direita ou da direita para a esquerda. O material destaca que operadores aritméticos comuns normalmente possuem associatividade à esquerda, enquanto a exponenciação pode possuir associatividade à direita.

O exemplo $-2 ** 2$ foi especialmente útil porque mostra que linguagens diferentes podem interpretar a mesma escrita de maneira distinta. Em Python, o resultado é -4. Java não possui o operador $**$. Em JavaScript, essa combinação exige atenção à regra da linguagem.

Também foi apresentado o uso de notação prefixa em Lisp, na qual os parênteses indicam a estrutura da expressão, e a avaliação da direita para a esquerda em APL. A principal conclusão foi que, quando houver dúvida, o uso de parênteses torna a intenção explícita e facilita a manutenção.

Expressões aritméticas: questões de projeto

- Quais são as regras de **precedência**?
- Quais são as regras de **associatividade**?
- Em que **ordem os operandos** são avaliados?
- Há restrições a **efeitos colaterais**?
- O usuário pode **sobrecarregar** operadores?
- Que **mistura de tipos** é permitida?

Lisp dispensa a precedência (p. 306)
(+ a (* b c)): notação prefixa, e os parênteses dizem a ordem.

$a + b * c$ com $a=3$, $b=4$, $c=5$

da esquerda para a direita
 $(3 + 4) * 5 = 35$

da direita para a esquerda
 $3 + (4 * 5) = 23$

com precedência (* antes de +)
 $3 + 4 * 5 = 23$

APL: sempre da direita para a esquerda (p. 304)
 $3 \times 4 + 5 = 27$

Sebesta, 11ª ed. - §7.2 e §7.2.1 - p. 300-305

3. Precedência na prática e efeitos colaterais

A comparação entre Python, JavaScript e Java demonstrou que uma mesma sequência de símbolos pode produzir resultados diferentes. Foram observadas diferenças em exponenciação, concatenação de strings, divisão inteira e operador de resto.

A aula avançou então para a ordem dos operandos e os efeitos colaterais. Um efeito colateral ocorre quando uma função modifica alguma informação externa ao seu resultado. Isso pode fazer com que a ordem em que os operandos são avaliados altere o resultado final.

No exemplo apresentado, uma variável inicia com valor 5 e uma função altera essa variável para 17 e retorna 3. A expressão `a = a + fun1()` evidencia por que é importante saber como a linguagem determina a ordem de avaliação. Em Java, a ordem é definida da esquerda para a direita; em C, o padrão não estabelece a mesma garantia para o exemplo.

O conceito de transparência referencial também foi abordado. Funções sem efeitos colaterais são mais previsíveis, pois o resultado depende essencialmente dos argumentos. Isso facilita otimizações, testes e compreensão do código.

Ordem dos operandos e efeitos colaterais

Exemplo do livro (p. 307)

```
int a = 5;

int fun1(void) {
    a = 17;    /* efeito colateral */
    return 3;
}

int main(void) {
    a = a + fun1(); /* 8 ou 20? */
    printf("a = %d\n", a);
}
```

8

Java

a ordem é garantida: da esquerda para a direita (p. 308)

20

C (gcc 14)

a ordem não é definida pelo padrão

20

Go

a especificação não define a leitura de `a` em relação à chamada

Transparência referencial (p. 308): sem efeitos colaterais, a ordem deixa de importar, e $f(x) + f(x)$ pode virar $2 * f(x)$.

Sebesta, 11ª ed. · §7.2.2 · p. 307-309 · exemplos/02_efeitos_colaterais

4. Sobrecarga de operadores

Sobrecarga de operadores significa utilizar um mesmo símbolo para representar operações diferentes conforme o tipo ou o contexto. O operador + pode representar soma de inteiros ou reais e, em Java, também concatenação de strings.

O exemplo 'a' + 1 em Java produz 98 porque o caractere participa de uma operação numérica nesse contexto. Já "" + 'a' + 1 resulta em a1, pois a string inicial faz com que a expressão seja tratada como concatenação.

O material também mostrou a sobrecarga definida pelo usuário. Python permite implementar métodos especiais como `__add__` e `__mul__`, possibilitando que objetos próprios respondam aos operadores. O exemplo de Vetor mostrou como `v1 + v2` pode produzir um novo vetor e como `v * 3` pode multiplicar suas coordenadas.

A sobrecarga pode tornar o código mais natural, mas também pode gerar confusão quando um símbolo possui significados muito diferentes. Por isso, o uso deve preservar a legibilidade e respeitar a expectativa de quem lê o programa.

Operadores sobrecarregados

- O mesmo símbolo com **vários significados**: + soma inteiros, reais e concatena (Java)
- Risco (p. 309): & em C++ é E bit a bit e "endereço de"
- Definida pelo usuário (p. 404): C++, C#, Python, Rust, Kotlin. Java **não** permite

```
Java
System.out.println('a' + 1);    // 98
System.out.println("" + 'a' + 1); // a1
x.add(y).multiply(tres);    // BigDecimal: sem +
```

Python - sobrecarga com métodos especiais

```
class Vetor:
    def __init__(self, x, y):
        self.x, self.y = x, y

    def __add__(self, outro):
        # v1 + v2
        return Vetor(self.x + outro.x,
                      self.y + outro.y)

    def __mul__(self, k):
        # v * 3
        return Vetor(self.x * k, self.y * k)

v = Vetor(1, 2) + Vetor(3, 4)    # Vetor(4, 6)
print(v * 3)                    # Vetor(12, 18)
```

Sebesta, 11ª ed. - §7.3 - p. 309-310 e §9.11 - p. 404 - exemplos/03_sobrecarga

5. Conversões de tipos: alargamento, estreitamento e coerção

Conversões de tipos foram estudadas como um ponto fundamental para entender expressões. Alargamento ocorre quando um valor passa para um tipo com maior capacidade de representação. Estreitamento ocorre quando a conversão pode perder informação.

Em Java, um int pode ser convertido implicitamente para float. Entretanto, o exemplo da aula demonstrou que essa conversão pode provocar perda de precisão: o valor 123456789, ao passar por float e retornar para int, pode resultar em 123456792.

Também foi apresentado o cast explícito. Converter 3.99 para int produz 3, enquanto converter 200 para byte produz -56. Outro detalhe importante é que operações com byte em Java podem ser promovidas para int. Assim, byte $s = b + c$ pode gerar erro.

A divisão $1 / 2$ resulta em 0 porque ambos os operandos são inteiros. Já $1 / 2.0$ resulta em 0.5. A linguagem considera os tipos dos operandos ao determinar a operação.

Conversões de tipos: alargamento, estreitamento e coerção

Java · coerção e cast

```
int grande = 123456789;
float f = grande;      // alargamento implícito
System.out.println((int) f); // 123456792 !

double d = 3.99;
System.out.println((int) d); // 3 (cast)
System.out.println((byte) 200); // -56

byte b = 10, c = 20;
// byte s = b + c; // erro: b + c é int (p. 312)

System.out.println(1 / 2); // 0
System.out.println(1 / 2.0); // 0.5
```

Linguagem	int → real (implícito)	int32 + int64
C	sim	coerção
Java	sim (alargamento)	coerção para long
Python	sim	(um só tipo int)
Go	não: float64(i)	erro de compilação
Rust	não: i as f64	erro de compilação
ML, F# (p. 312)	não	erro de compilação

Coerção esconde erros (p. 312): em $d = b * a$; com a digitado por engano no lugar de c, o Java converte e compila.

Sebesta, 11ª ed. - §7.4 - p. 310-313 - exemplos/04_conversoes

6. Coerção implícita em JavaScript e PHP

JavaScript foi usado como exemplo de linguagem que realiza várias conversões implícitas.

"5" - 2 resulta em 3, enquanto "5" + 2 resulta em "52". A diferença ocorre porque o operador - exige comportamento numérico, enquanto + também pode realizar concatenação.

Outros exemplos apresentados foram true + 1, null + 1 e undefined + 1. O resultado desses casos demonstra que valores podem ser convertidos automaticamente para números ou produzir NaN.

A comparação com == foi um dos pontos mais importantes. 0 == "" resulta em verdadeiro, "" == "0" resulta em falso e 0 == "0" resulta em verdadeiro. Já 0 === "0" resulta em falso porque === não realiza coerção.

Em PHP, o type juggling também permite comparações que podem surpreender. A recomendação prática apresentada foi preferir === e !== quando a intenção for comparar sem coerção automática. Isso reduz ambiguidades e torna a intenção do código mais clara.

Coerção implícita: JavaScript e PHP

JavaScript

```
"5" - 2           // 3
"5" + 2           // "52"
"5" * "2"         // 10
true + 1          // 2
[] + {}           // "[object Object]"
null + 1          // 1
undefined + 1     // NaN
"b" + "a" + "a" + "a" // "baNaNa"

0 == ""          // true
"" == "0"        // false
0 == "0"         // true (não é transitivo!)
0 === "0"        // false (=== não converte)
```

PHP - "type juggling"

```
var_dump("0e123" == "0e456"); // true !
var_dump("0e123" === "0e456"); // false
var_dump("1" == "01"); // true

md5("240610708") // "0e462097431906..."
md5("QNKCDZO") // "0e830400451993..."
// == entre os dois hashes: true
```

Regra prática

Em JavaScript e PHP, compare com === e !==, que não fazem coerção.

Sebesta, 11ª ed. - §7.4.1 Coerção em expressões - p. 311-312 - exemplos/04_conversoes

7. Expressões booleanas e curto-circuito

As linguagens possuem diferentes formas de tratar valores booleanos. Em C, zero representa falso e qualquer valor diferente de zero pode representar verdadeiro. Por isso, uma expressão como `if (-1)` é considerada verdadeira.

O exemplo `3 > 2 > 1` em C demonstrou uma armadilha importante. Primeiro, `3 > 2` resulta em 1. Depois, `1 > 1` resulta em falso. Em Python, a mesma construção é uma comparação encadeada e representa a verificação conjunta das relações.

O curto-circuito foi estudado principalmente com `&&`. Em Java, a segunda parte de uma expressão com `&&` pode deixar de ser avaliada quando a primeira parte já determina que o resultado será falso. Isso pode evitar acessos inválidos e exceções.

Também foram apresentados `?.` e `??` em JavaScript. O operador `?.` permite acessar propriedades sem provocar erro quando um objeto intermediário é nulo. O operador `??` substitui somente `null` ou `undefined`, preservando valores como 0.

Avaliação em curto-circuito

Java · `&&` faz curto-circuito, & não

```
// busca do livro (p. 316)
while (i < lista.length && lista[i] != chave)
    i++;           // && evita lista[lista.length]

String s = null;
s != null && s.length() > 0 // false
s != null & s.length() > 0 // NullPointerException
```

JavaScript · `?.` e `??`

```
const u = { nome: "Ana", endereco: null };
u.endereco?.cidade // undefined, sem erro

const qtd = 0;
qtd || 10           // 10 (o 0 se perdeu!)
qtd ?? 10           // 0  (só troca null/undefined)
```

Cuidado com efeitos colaterais (p. 316)

Em `(a > b) || (b++ / 3)`, o `b++` só acontece quando `a <= b`. Em Python, `0 or "padrão"` devolve "padrão": and e or devolvem um dos **operandos**.

Sebesta, 11ª ed. - §7.6 - p. 315-317 - exemplos/05_booleanas

8. Sentenças de atribuição

A atribuição pode possuir características diferentes em cada linguagem. Em C, C++ e JavaScript, ela pode funcionar como uma expressão. Isso permite construções como `sum = count = 0`.

Uma armadilha clássica é `if (x = y)`, em linguagens que aceitam atribuição dentro da expressão condicional. O programador pode ter pretendido comparar `x` e `y`, mas escreveu uma atribuição. O código pode compilar e produzir comportamento inesperado.

Java possui uma diferença importante: a atribuição é uma expressão, mas o `if` exige um valor booleano. Python não possui a atribuição tradicional como expressão, embora ofereça o operador morsa `:=` para casos específicos.

Também foi estudada a atribuição múltipla em Python, como `a, b = b, a`, que permite trocar valores sem uma variável temporária. Em linguagens funcionais, construções como `let` e `val` podem dar nomes a valores de maneira diferente da atribuição imperativa tradicional.

Sentenças de atribuição

C · a atribuição é uma expressão (p. 319-320)

```
sum = count = 0;           // múltiplos alvos
a = b + (c = d / b) - 1;    // exemplo do livro

while ((ch = getchar()) != EOF) { ... }

if (x = y) { ... }          // compila! atribui e testa y
```

Python · atribuição múltipla (p. 320)

```
a, b = b, a                # troca sem temporária
if (n := len(dados)) > 5: ... # morsa (Python 3.8+)
```

Linguagem	Atribuição é expressão?
C, C++, JavaScript	sim: <code>if (x = y)</code> compila
Java, C#	sim, mas o <code>if</code> exige booleano
Python	não, exceto o operador morsa <code>:=</code>
Go	não: nem <code>x++</code> é expressão
Rust	vale <code>()</code> : <code>if x = 5</code> não compila

Linguagens funcionais (p. 321)

`val` (ML) e `let` (F#, Rust) dão nome a um valor, que não muda.

Sebesta, 11ª ed. · §7.7 · p. 317-321 · exemplos/06_atribuicao

9. Atribuição de modo misto e estruturas fundamentais de controle

Na atribuição de modo misto, o programa combina valores de tipos diferentes. Java permite determinadas conversões de alargamento, como atribuir um int a um double, mas exige cast explícito quando há estreitamento.

O material destacou que operadores compostos podem esconder conversões. Também apresentou a armadilha do cálculo de porcentagem: se `acertos` e `total` forem inteiros, `acertos / total` realiza divisão inteira antes de o resultado ser multiplicado por 100 e armazenado em `double`.

Em seguida, foram apresentadas as três estruturas fundamentais de controle: sequência, seleção e iteração. Sequência executa comandos em ordem; seleção escolhe entre caminhos; iteração repete comandos enquanto uma condição for satisfeita.

O princípio de Böhm e Jacopini mostra que essas estruturas são suficientes para expressar algoritmos, enquanto outras construções, como `for` e `switch`, existem principalmente para facilitar a escrita e melhorar a legibilidade.

Estruturas de controle: quantas são necessárias?

Sequência	Seleção	Iteração
uma sentença após a outra	escolher um caminho	repetir enquanto uma condição vale
<pre>a = 1; b = a + 2;</pre>	<pre>if (x > 0) ... else ...</pre>	<pre>while (x > 0) x--;</pre>

Böhm e Jacopini (1966): sequência, seleção e laço lógico bastam para qualquer algoritmo. O goto não é necessário.

Por que então tantas outras (`for`, `switch`, `do`, `foreach`)? **Facilidade de escrita.** O limite é a **legibilidade**: quanto maior a linguagem, menos do código alheio você reconhece (p. 329).

Sebesta, 11ª ed. - §8.1 - p. 328-330

10. Seleção: if, else e switch

O chamado else pendente foi estudado para demonstrar que a indentação pode enganar. Em C, Java, C++ e C#, o else pertence ao if mais próximo que ainda não possui um else.

Portanto, a organização visual do código não muda essa regra.

O uso de chaves reduz esse risco e torna a intenção explícita. Algumas linguagens adotam regras ainda mais rígidas para blocos, enquanto outras, como Python, utilizam a própria estrutura sintática para delimitar a associação.

Na seleção múltipla, o material mostrou o switch tradicional de C e o problema causado pela ausência de break. Quando o break não é utilizado, a execução pode continuar nos casos seguintes. Java moderno oferece switch como expressão e utiliza setas para evitar esse comportamento.

Rust apresenta match, que exige que as possibilidades sejam cobertas. Esse requisito de exaustividade é uma característica que pode ajudar a detectar casos não tratados durante a compilação.

Seleção múltipla: do switch de C ao match

C · sem break, o controle "cai" (p. 335)

```
switch (index) {
  case 1:
  case 3: odd += 1;
  case 2:
  case 4: even += 1;
  default: printf("Error\n");
}
// index = 1: odd, even E erro!
```

Java 14+ · switch como expressão

```
String tipo = switch (dia) {
  case SAB, DOM -> "fim de semana";
  case SEG, TER, QUA,
  QUI, SEX -> "dia útil";
}; // sem break, sem cair;
// exaustivo para enum
```

Rust · match exaustivo

```
match n {
  i32::MIN..=-1 => "negativo",
  0 => "zero",
  1..=9 => "um dígito",
  _ => "vários dígitos",
}
// sem o _ : não compila
```

Go: cada caso termina sozinho, e cair exige fallthrough. C#: cair implicitamente é proibido (p. 336). Python 3.10: match com padrões e guardas.

Sebesta, 11ª ed. - §8.2.2 - p. 334-341 - exemplos/07_selecao

11. Laços e iteração

O for foi analisado como uma forma compacta de organizar inicialização, condição e atualização. Em C, a variável de controle pode ser modificada dentro do corpo, alterando a sequência percorrida. Em Python, o for percorre um iterável; alterar a variável de iteração não altera a sequência produzida por range.

Outro ponto estudado foi a diferença entre for...in e for...of em JavaScript. O primeiro percorre propriedades ou índices de um objeto, enquanto o segundo percorre os valores de um iterável. Confundir os dois pode produzir resultados incorretos ao trabalhar com arrays.

A aula também mostrou o risco de um laço sem progresso. O exemplo dos tocadores Zune demonstrou que uma condição pode permanecer verdadeira se nenhuma variável relevante for alterada. O do-while foi lembrado como uma estrutura de pós-teste, na qual o corpo executa pelo menos uma vez.

Esses exemplos reforçaram que a análise de um laço deve considerar condição de entrada, alteração das variáveis e condição de saída.

Laços controlados por contador

C - o for é um laço lógico disfarçado

```
for (int i = 0; i < 10; i++) {  
    printf("%d ", i);  
    i += 2;           // altera a contagem  
}                    // 0 3 6 9  
  
for (; i <= 100; ) // expressões opcionais  
    soma += i++;
```

Python - o for percorre um iterável

```
for i in range(0, 10):  
    print(i, end=" ")  
    i += 2           # NÃO muda nada  
                    # 0 1 2 3 4 5 6 7 8 9  
  
for i in range(10, 0, -3): # 10 7 4 1  
    ...
```

Questões de projeto (p. 342)

Qual é o tipo e o escopo da variável? Posso alterá-la no corpo? Os parâmetros são avaliados uma vez ou a cada volta?

Sebesta, 11ª ed. - §8.3.1 - p. 342-345 - exemplos/08_iteracao

12. Goto, comandos protegidos e atividade de fixação

O goto realiza um desvio incondicional. A aula relacionou seu uso ao caso “goto fail”, mostrando como um salto duplicado pode impedir que uma etapa importante seja alcançada. O material citou a discussão de Dijkstra sobre o uso de goto e apresentou break, continue e return como formas de realizar determinados desvios de maneira mais restrita.

Também foram apresentados os comandos protegidos de Dijkstra, baseados em guardas. Quando mais de uma guarda é verdadeira, pode haver escolha não determinística. O conceito foi relacionado ao select de Go, em que múltiplas alternativas podem estar prontas.

Na atividade final, a turma deveria encontrar bugs em seis trechos de código. Entre os problemas estavam o uso de for...in em array, o emprego de d or 10 em Python, um switch sem break, um ponto e vírgula após if em C, uma questão de escopo em Go e uma divisão inteira em Java.

A atividade consolidou a ideia de que encontrar um bug exige mais do que descobrir que o resultado está errado: é necessário identificar a regra da linguagem responsável pelo comportamento e propor uma correção coerente.

Desvio incondicional: o goto

“goto fail” (Apple, 2014) - versão simplificada

```
int valida_certificado(void) {
    int err;
    if ((err = verifica_hash()) != 0)
        goto fail;
    goto fail;           // duplicada: desvia SEMPRE
    if ((err = verifica_assinatura()) != 0)
        goto fail;       // nunca é alcançada
fail:
    return err;           // 0 = "certificado válido"
}
```

Dijkstra (1968)

“Go To Statement Considered Harmful”

Quem tem goto hoje

C, C++, C#, Go. Java **não** tem (p. 353)

Gotos restritos

break, continue e return melhoram a legibilidade (p. 353)

Com chaves obrigatórias (Go, Rust) ou com gcc -Wall (aviso de endentação enganosa), o bug teria sido pego.

Sebesta, 11ª ed. - §8.4 - p. 352-353 - exemplos/09_desvio_protegido

13. Conclusão e principais aprendizados

A Aula 07 mostrou que expressões, atribuições e estruturas de controle são muito mais do que elementos básicos de sintaxe. Cada linguagem possui decisões próprias sobre precedência, associatividade, conversões, tipos booleanos, ordem de avaliação e mecanismos de controle.

O principal aprendizado foi a necessidade de prever o comportamento do código antes de simplesmente assumir que ele funcionará como parece. Os exemplos de Python, Java, JavaScript, C, Go e Rust mostraram que uma mesma ideia pode ser implementada e interpretada de maneiras diferentes.

Também ficou evidente a importância da legibilidade. Parênteses explícitos, conversões conscientes, uso correto das estruturas de seleção, escolha adequada entre tipos de laço e cuidado com efeitos colaterais tornam o programa mais previsível e mais fácil de manter.

A aula ainda mostrou que regras aparentemente pequenas podem ter consequências importantes, principalmente em segurança e confiabilidade. Os exemplos históricos de “goto fail”, do Zune e das comparações com coerção em PHP ajudaram a relacionar a teoria com problemas reais.

Como resultado, o conteúdo contribuiu para uma compreensão mais profunda do funcionamento das linguagens de programação e para uma postura mais cuidadosa na leitura, escrita e depuração de código.

Síntese: armadilhas e como as linguagens ajudam

Tema	Armadilha	Como as linguagens ajudam
Precedência	<code>-2 ** 2, 1 + 2 + "3"</code>	parênteses; JS obriga <code>(-2) ** 2</code>
Operandos	<code>a + f()</code> com efeito colateral (C, Go)	Java fixa a ordem; funções puras
Conversões	coerção silenciosa (C, JS, PHP)	Go e Rust exigem conversão explícita; <code>==</code>
Booleanos	<code>0, "", []</code> são falsos; <code>a > b > c</code> em C	if só com booleano (Java, Go, Rust); <code>??</code>
Atribuição	<code>if (x = y)</code>	atribuição não é expressão (Python, Go, Rust)
Seleção	switch sem break; else pendente	<code>-></code> , match exaustivo, chaves obrigatórias
Laços	<code>for...in</code> no JS; laço sem progresso	<code>for...of</code> , iteradores, break rotulado
Desvio	goto sem restrição	break, continue, return: gotos restritos

Sebesta, 11ª ed. - Caps. 7 e 8 - p. 299-361

Referência: SEBESTA, Robert W. Conceitos de Linguagens de Programação. 11. ed. Capítulos 7 e 8, p. 299-361. Material da Aula 07 – Expressões, Atribuição e Controle.